

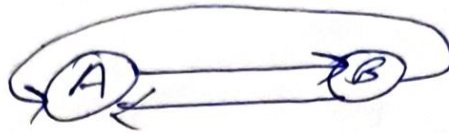
4/11/2025
Tuesday

Fibonacci Heap

- It is based on Fibonacci series

0 1 1 2 3 5

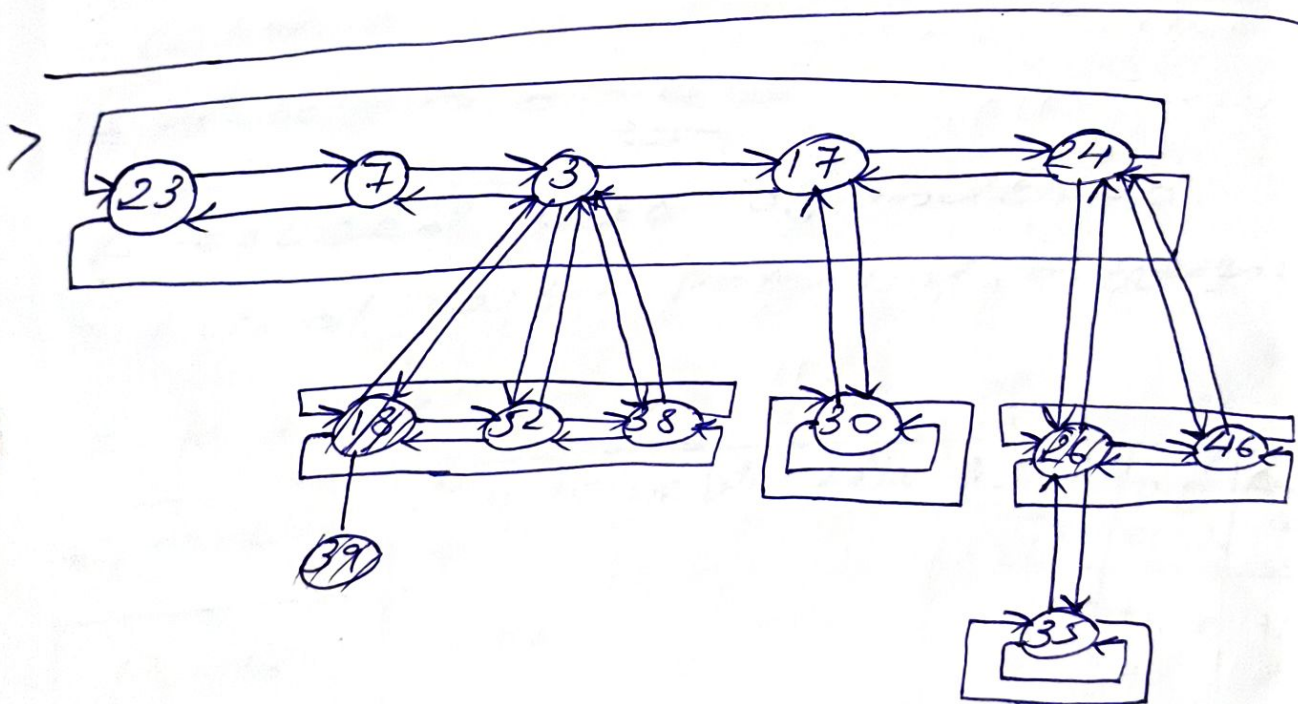
- It is circular doubly linked list



- It is min-heap ordered structure

- Total no. of trees in the fibonacci heap
= $t(H)$

- $m(H) \rightarrow$ marked nodes



A fibonacci heap is a collection of rooted trees that are min-heap ordered, i.e., key of any node is greater than or equal to its parent.

Fibonacci heap follows "circular doubly linked list" structure.

Each node 'x' contains a pointer $x.p$ to its parent and a pointer $x.child$ to any of its children.

Ex: - 30 has a pointer $x.p$ to its parent 17

Children of 'x' are linked together in a circular doubly linked list manner.

Two advantages of circular doubly linked used in Fibonacci heap are:-

① We can insert into any location or remove a node from anywhere in circular doubly linked list of time $O(1)$.

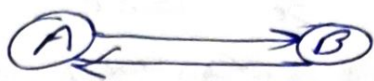
Ex:- in normal linked list, we need to traverse from 1st to end & then insert
 } $O(n)$. Here, direct insert $O(1)$

② On inserting two such lists into one circular doubly linked list of time $O(1)$.

> Properties of Fibonacci Heap

① The trees are not necessarily binomial heap (In binomial heap, $B_0 = 2^0$, $B_1 = 2^1 = 2$ nodes, $B_2 = 2^2 = 4$ nodes, $B_3 = 2^3 = 8$ nodes, ... There is no such thing as B_4 .)

② Siblings are bidirectionally linked



③ There is a pointer $\text{min}(H)$ or $H.\text{min}$ to the root with minimum key.

④ The root degrees are not unique.

⑤ A special attribute $n(H)$, meaning total number of nodes.

⑥ $t(H)$ means number of trees in rootlist of H .

⑦ $m(H)$ = number of marked nodes in Fibonacci heap, H .

> Construction of Fibonacci Heap

Rule

For a node of degree ' k ', the minimum number of nodes in its subtree is given by

$$N(k) \geq F_{k+2}$$

deg (k)	k+2	N _k
0	$F_{0+2} = F_2$	1 node
1	$F_{1+2} = F_3$	2 nodes
2	$F_{2+2} = F_4$	3 nodes
3	$F_{3+2} = F_5$	5 nodes
4	$F_{4+2} = F_6$	8 nodes
5	$F_{5+2} = F_7$	13 nodes

7 Fibonacci series

0	→ F_0	21 → F_8
1	→ F_1	34 → F_9
1	→ F_2	55 → F_{10}
2	→ F_3	
3	→ F_4	
5	→ F_5	
8	→ F_6	
13	→ F_7	

6	$F_{6+2} = F_8$	21 nodes
7	$F_{7+2} = F_9$	34 nodes
8	$F_{8+2} = F_{10}$	55 nodes

Next, we can note that:-

$$N(k) \geq F_{k+2}$$

> Potential Function

To analyze the performance of fibonacci heap operations, we use potential method. Potential function of fibonacci heap is:-

$$\Phi(H) = t(H) + 2m(H)$$

where,

$t(H)$ = Number of trees in root list of H

$m(H)$ = Number of marked nodes in Fibonacci heap, H .

* An amortized analysis:-

$$\begin{aligned} \text{Amortized cost} &= \text{Actual cost} + \text{potential difference} \\ &= \text{Actual cost} + \Phi(H) \end{aligned}$$

$$\Rightarrow \text{Amortized cost} = \text{Actual cost} + t(H) + 2m(H)$$

. An the gives example:-

$$\Phi(H) = 5 + 2 \times 3 = 5 + 6 = \underline{\underline{11}}$$

Operations in Fibonacci Heap

① Creating a new fibonacci heap

② Inserting a node $\rightarrow O(1)$

③ Finding minimum node $\rightarrow O(1)$

④ Union of two fibonacci heap $\rightarrow O(1)$

⑤ Extract minimum node $\rightarrow O(\log n)$

⑥ Decreasing a key $\rightarrow O(1)$

⑦ Deleting a node $\rightarrow O(\log n)$

① Creating a new Fibonacci Heap

To make an empty Fibonacci heap, MAKE_FIB_HEAP procedure allocates memory and returns a fibonacci heap object, H, where

* Here, only memory is allocated, no value is allocated to it.
 ~~so, no value, no min~~

$$H.n = 0$$

$$H.min = NIL$$

There are no trees in H.

$$\text{So, } T(H) = 0$$

$$m(H) = 0$$

and the potential function of empty tree, $\phi(H) = 0$

> ② Inserting a node

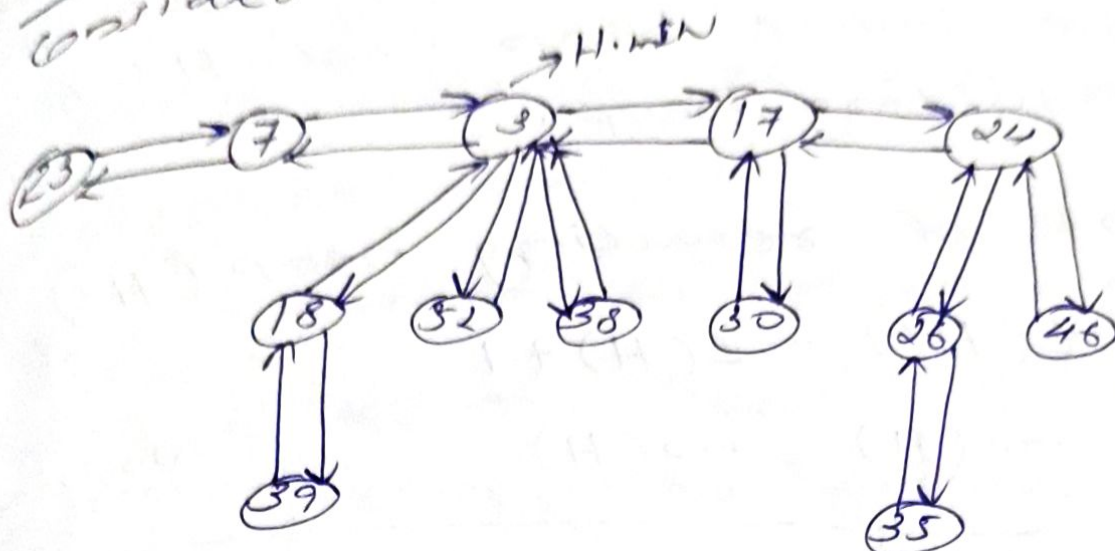
A node can be inserted using $\text{FIB_HEAP_Insert}(H, x)$ where,
 $H = \text{Heap}$
 $x = \text{new node}$

ALGORITHM

$\text{FIB_HEAP_INSERT}(H, x)$

- 1) $x.\text{degree} = 0$
- 2) $x.p = \text{NIL}$
- 3) $x.\text{child} = \text{NIL}$
- 4) $x.\text{MARK} = \text{FALSE}$
- 5) If $H.\text{MIN} = \text{NIL}$
- 6) Create a rootlist containing just x
- 7) $H.\text{MIN} = x$
- 8) else insert x into H 's rootlist
- 9) If $x.\text{key} < H.\text{MIN}$
- 10) $H.\text{MIN} = x$
- 11) $H.n = H.n + 1$

Example :-
consider a heap :-



Insert a node 21 into heap H.

Ans:-

Firstly, create a node 21.

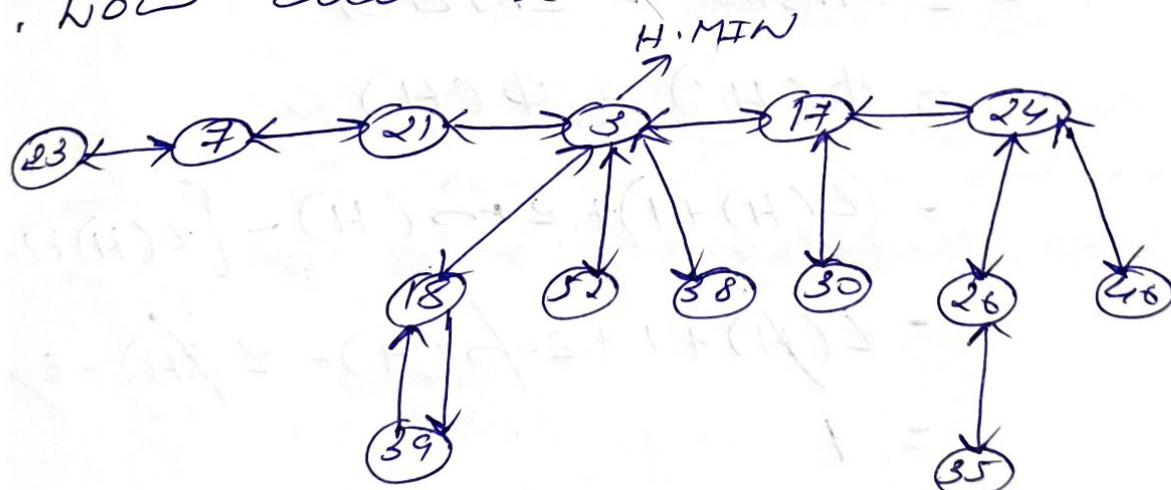
$x.degree = 0$

$x.p = NIL$

$x.child = NIL$

$x.MARK = FALSE$

Since, $H.MIN \neq NIL$,
Now add x into H 's root list



Since, $x.key > H.MIN$
do not need to change $H.MIN$

$H.in = H.in + 1$

• To determine the amortized cost of FIB-HEAP-INSERT, let 'H' be the input heap and H' be the resulting heap.

• P.F of resulting heap (H'),

$$\ell(H') = \ell(H) + 1$$

$$m(H') = m(H)$$

3rd node
inserted
6000

i.e., $\boxed{\phi(H') = (\ell(H) + 1) + 2m(H)}$

• P.F of initial heap (H),

$$\ell(H) = \ell(H), m(H) = m(H)$$

i.e., $\boxed{\phi(H) = \ell(H) + 2m(H)}$

before
inserting

∴ Potential function,

$$P.F = \text{Final} - \text{Initial}$$

$$= \phi(H') - \phi(H)$$

$$= (\ell(H) + 1) + 2m(H) - [\ell(H) + 2m(H)]$$

$$= \cancel{\ell(H)} + 1 + 2\cancel{m(H)} - \cancel{\ell(H)} - 2\cancel{m(H)}$$

$$= 1$$

$$\Rightarrow \underline{\underline{P.F = O(1)}}$$

Actual cost of inserting

- 1) Creating a node
- 2) Add node to the left of min
- 3) Update min

Time required to create = $O(1)$

Time required to add left = $O(1)$

Time required to update min = $O(1)$

$$\text{Actual cost} = O(1) + O(1) + O(1)$$

$$= O(3)$$

$$= \underline{\underline{O(1)}}$$

$\therefore$ Amortized cost = Actual cost + change in potential

$$= O(1) + O(1) = O(2)$$

$$= \underline{\underline{O(1)}}$$

Question

Insert a new node into the heap and find the amortized cost.

5/11/2025
Wednesday. → ③ Extract - Min

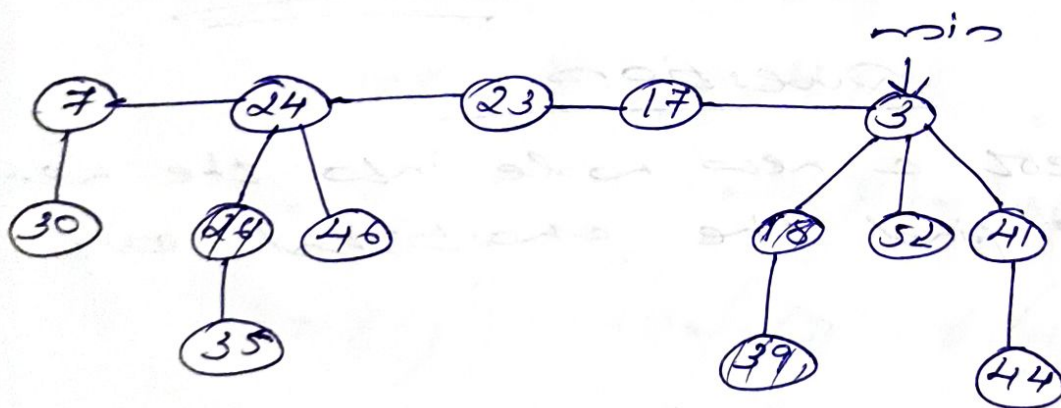
Step 1:- The first step is delete min; meld its children into root list; update min (next of current min).

• If it is last node, the next min pointer will be the starting node of the fibonacci heap because fibonacci heap maintains circular doubly linked list.

Step 2:- Consolidate trees so that no two roots have same degree.

Example

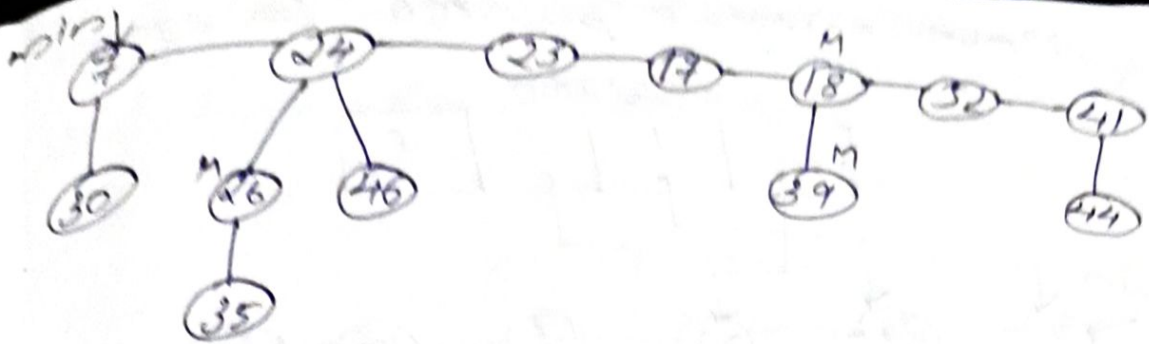
Consider the following fibonacci heap:-



Step 1:-

Ans:- Delete min (3).

- Meld the children of 3 into root list
- Update min → Here 3 is the last node of root list. So, 7 will become the new min.

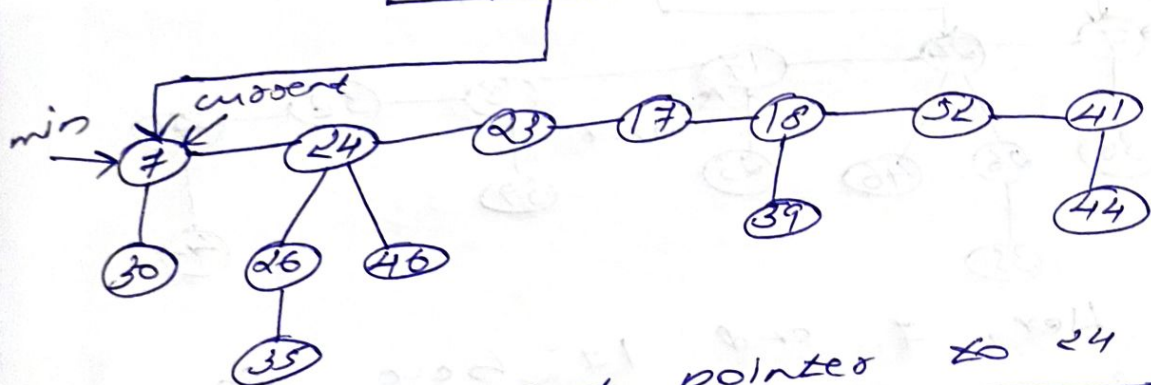


Step 2 :- Consolidate trees so that no two roots have same degree

Here, in the original heap, the maximum degree of root node, $D(n) = 3$

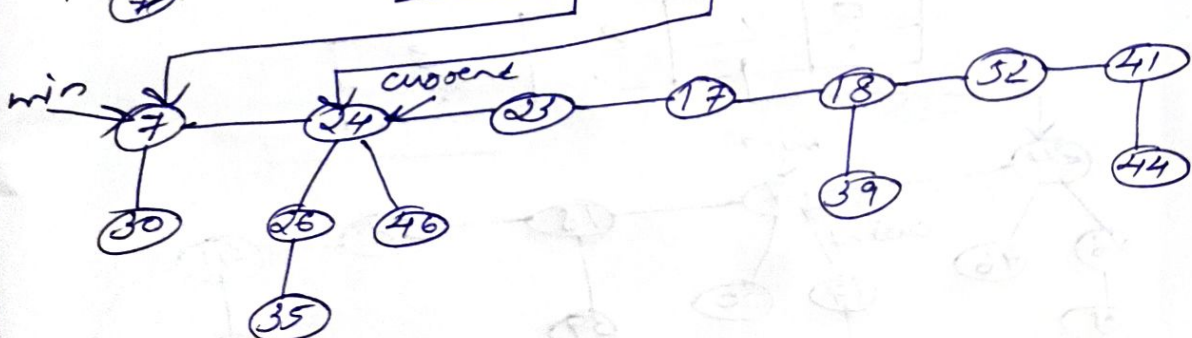
So, we need to create an array with indexes from 0 to 3.

0	1	2	3
.	.	.	.

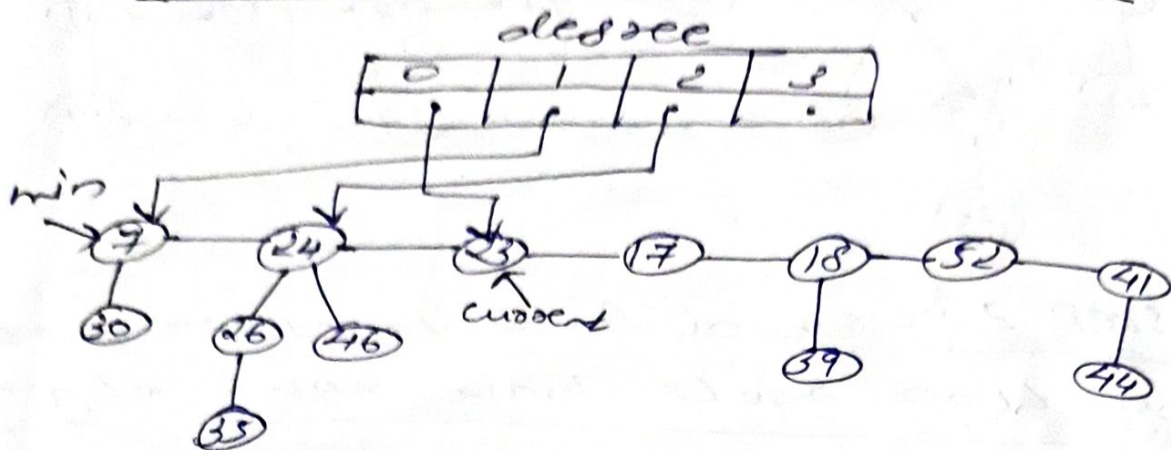


Move current pointer to 24

0	1	2	3
.	.	.	.

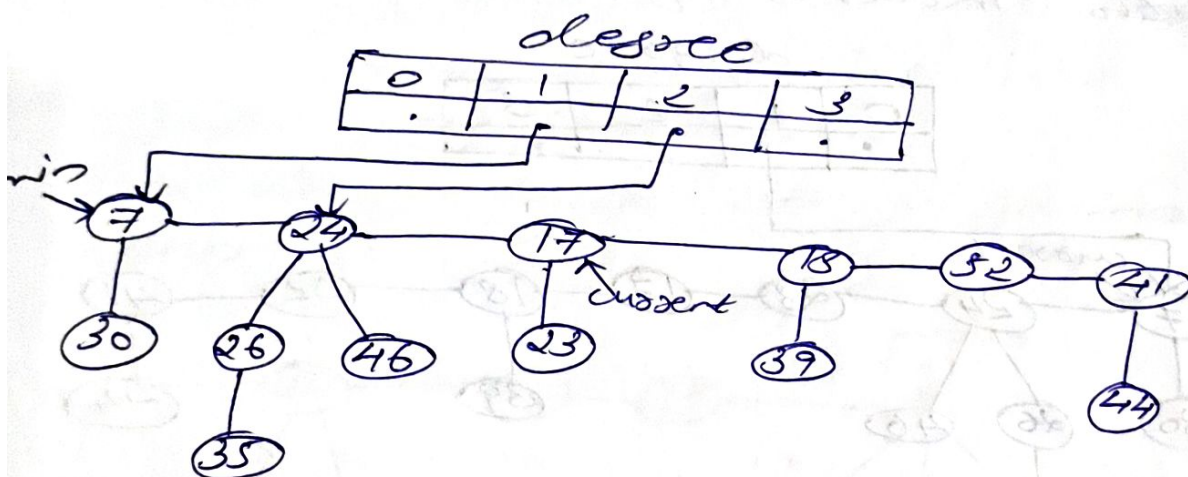


• Move current pointer to 23

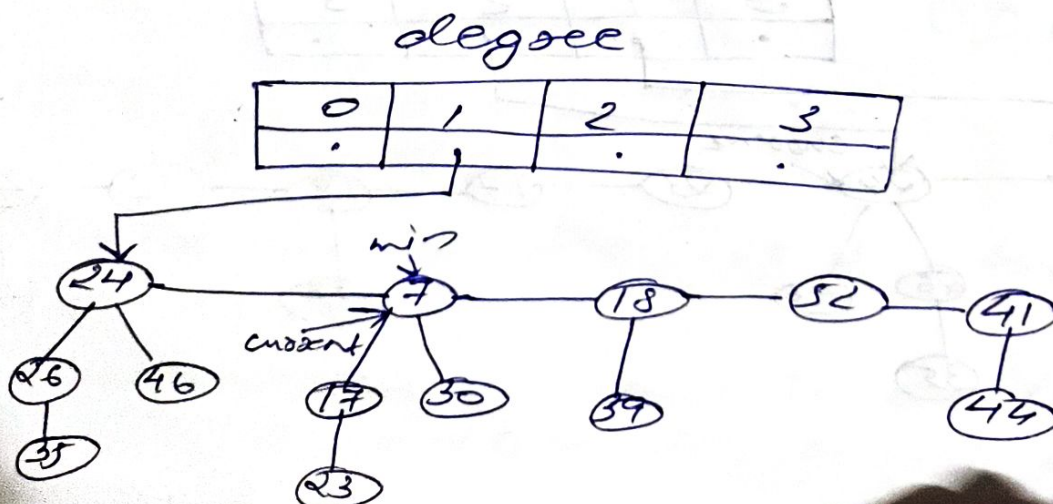


• Move current pointer to 17

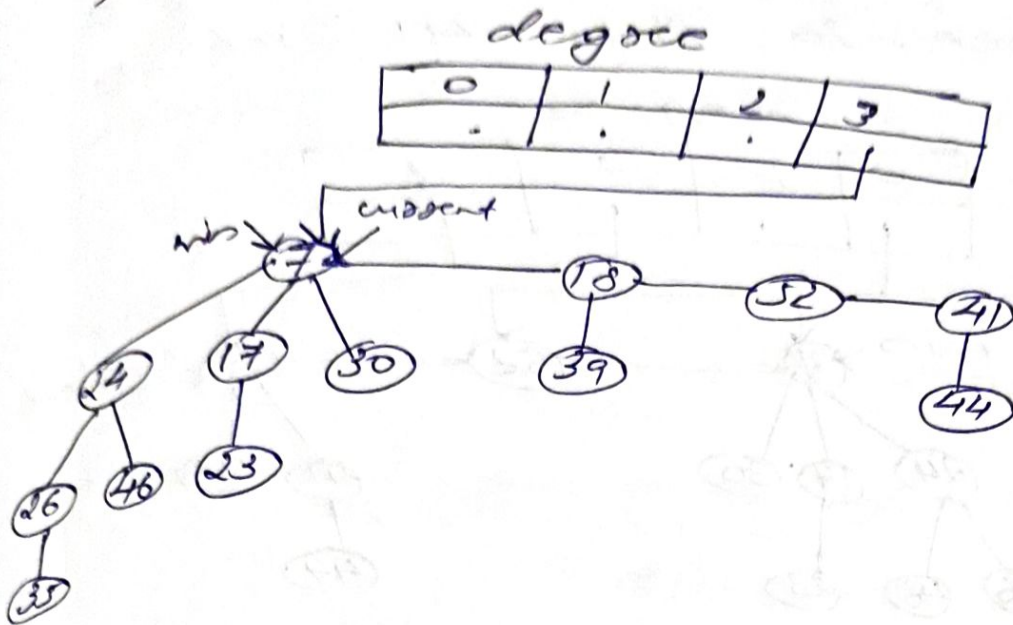
Here, 23 and 17 have degree 2, so, perform linking operation between 23 and 17.



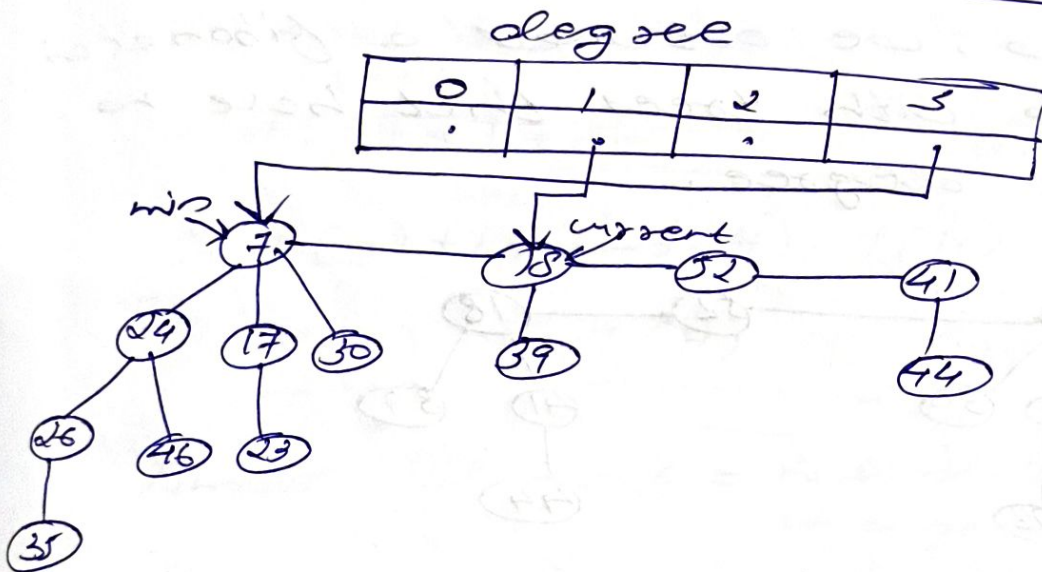
Here, 7 and 17 have degree 1, so, link 17 into 7.



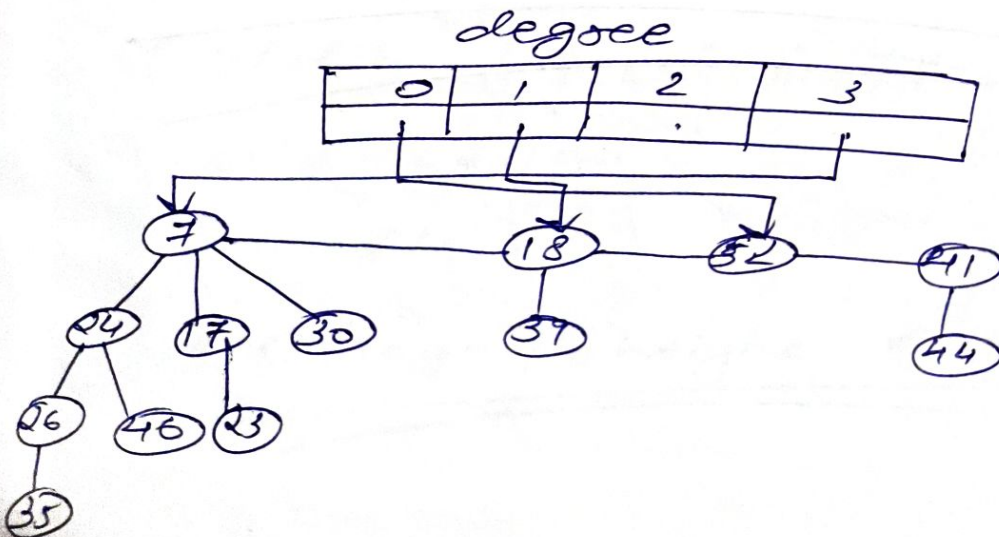
Here, 7 and 24 have same degree.
 So, link 24 into 7.



Move current pointer to 18

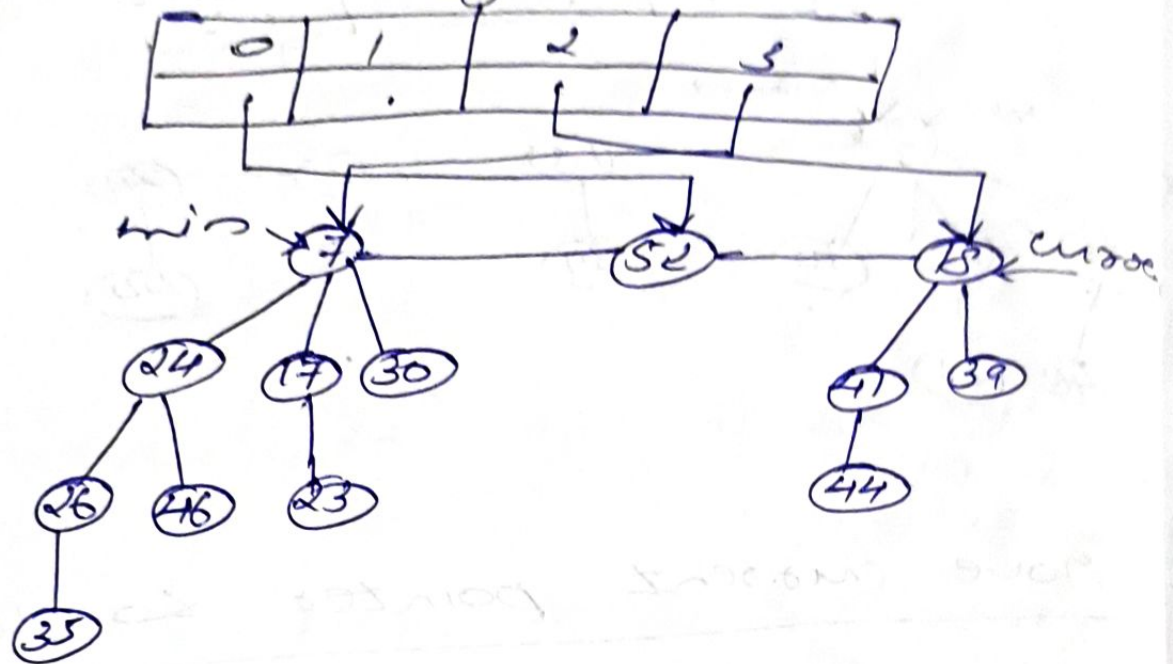


Move current pointer to 52

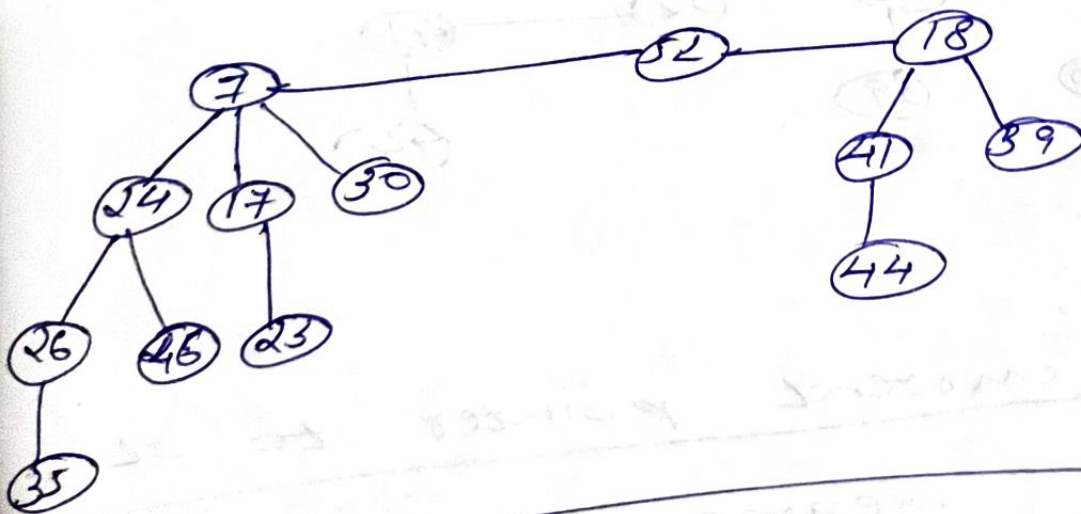


Move element pointed to 41

Here, 41 and 18 have same degree. So, link 41 into 18.



Now, we obtained a Fibonacci heap with trees that have no same degree.



> Amortized Analysis of extract min

$$\text{Actual cost} = O(d(n)) + O(T(H))$$

$$\begin{aligned} \text{At least total work done} \\ = O(d(n)) + T(H) \end{aligned}$$

$$\begin{aligned} \cdot \text{Potential function before} \\ = T(H) + 2m(H) \end{aligned}$$

$$\begin{aligned} \cdot \text{Potential function after extract min} \\ = d(n) + 1 + 2m(H) \end{aligned}$$

$$\cdot \text{Change in potential}$$

$$= \text{Final} - \text{Initial}$$

$$= d(n) + 1 + 2m(H) - [T(H) + 2m(H)]$$

$$= d(n) + 1 + 2m(H) - T(H) - 2m(H)$$

$$= d(n) + 1 - T(H)$$

$$\cdot \text{Amortized cost} = \text{Actual cost} + \text{change in potential}$$

$$= O(d(n)) + T(H) + d(n) + 1 - T(H)$$

$$= O(d(n)) + O(d(n)) + 1$$

$$= O(2d(n)) + 1$$

$$= O(d(n))$$

$$= O(\log n) = \text{height of tree}$$

> ④ Decrease Key

Not violated
↓
Just decrease the key

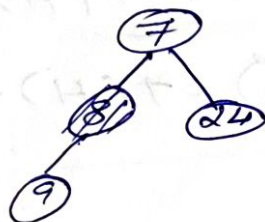
Violated
↓
cut tree rooted at x , meld into root Q , and unmark

Parent unmarked
↓
Mark parent

Parent marked
↓
do recursively for all ancestors that lose second child

> Marked node :- Has lost a child earlier.

Ex :-

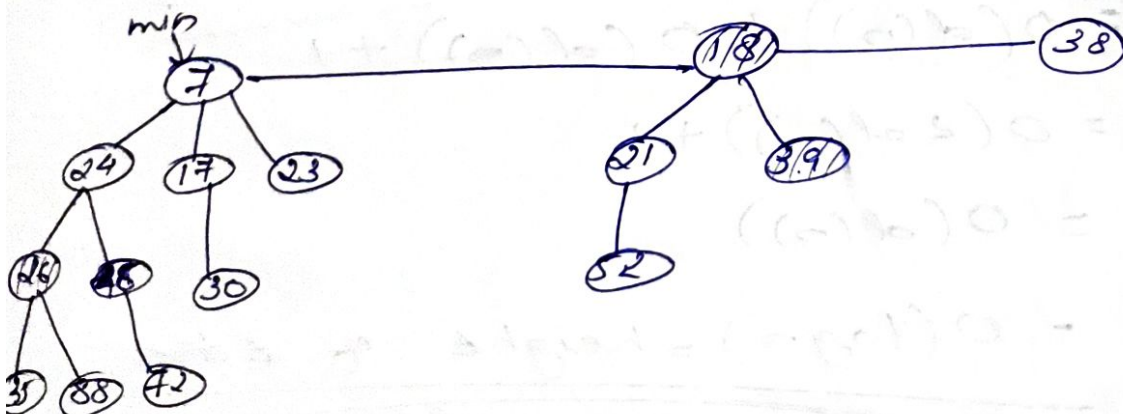


→ Here, 8 is marked.
It means 8's child is already cut.

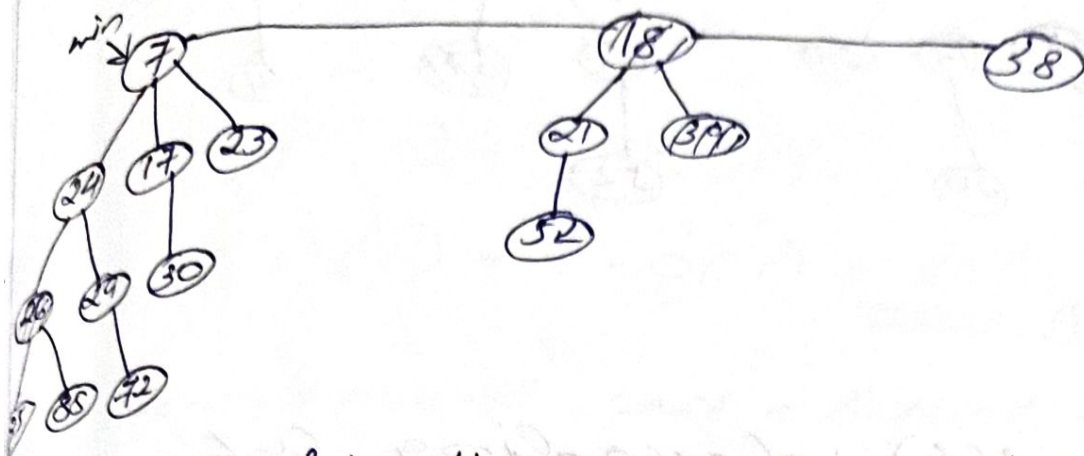
> Case 1 :- Heap order not violated

. decrease key of x .

. change heap min pointer (if necessary)



Next, decrease key of x from 46 to 29.



> Case 2: - Heap order violated

. Case 2(a): - Cut

① Decrease key of x

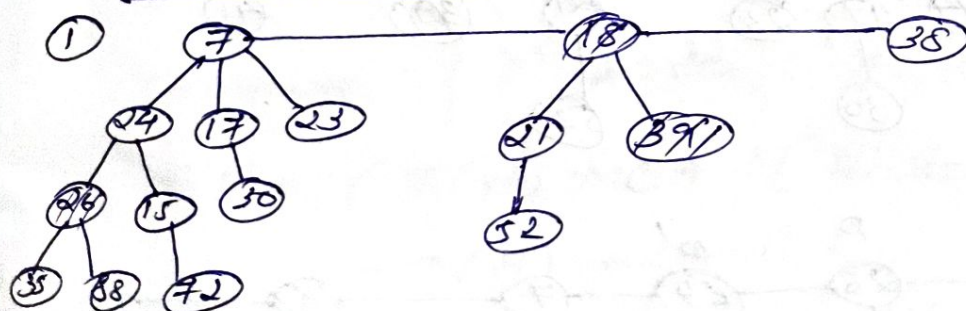
② Cut tree rooted at x , meld into root list and unmark.

2(a) ③ If parent p of x is unmarked (hasn't yet lost a child), mark it.

2(b) ④ Otherwise, cut p , meld into root list and unmark.

(and do so recursively for all ancestors that lose a second child).

. Decrease key from 29 to 15



7. Amortized Analysis

$$\begin{aligned}\text{Actual cost} &= \text{cost } O(1) + \text{recursive} \\ &\quad \text{cascadecut (c times, then } O(1) \text{ time needed)} \\ &= O(1) + c \cdot O(1) \\ &= O(1) + c \cdot O(1) = \underline{\underline{O(c)}}$$

$$\begin{aligned}\text{Amortized cost} &= \text{Actual cost} + \text{Change in potential initially} \\ &= T(H) + 2\ln(H)\end{aligned}$$

$$\text{After applying cost} = T(H) + c \times \text{edges}$$

$$\text{Masked nodes at most } M(H) - c + 2$$

{ Mathematically proven }

$$M(H) - c = \text{Each cascading unit, unmask all node}$$

$$\text{Change in potential} = \text{Final} - \text{Initial}$$

$$\text{Final} = \text{Initial}$$

$$\text{Final} = T(H) + c + M(H) - c + 2$$

Change in potential,

$$\begin{aligned}CP &= (T(H) + c + 2[M(H) - c + 2]) \\ &\quad - T(H) + 2\ln(H)\end{aligned}$$

$$= T(H) + c + 2\ln(H) - 2c + 4$$

$$= T(H) - 2\ln(H)$$

$$= -c + 4 = \underline{\underline{4 - c}}$$

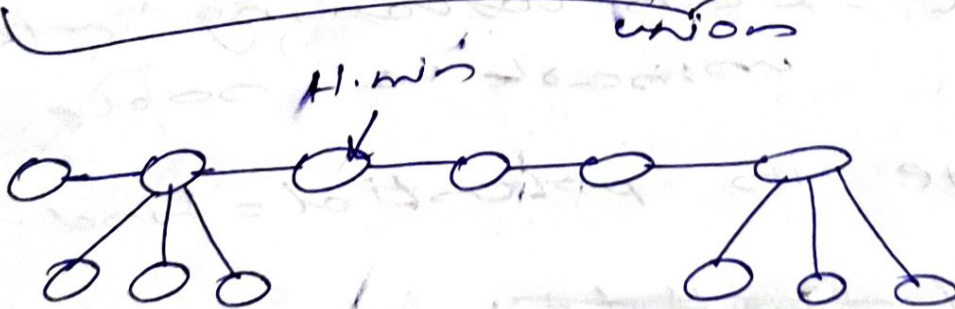
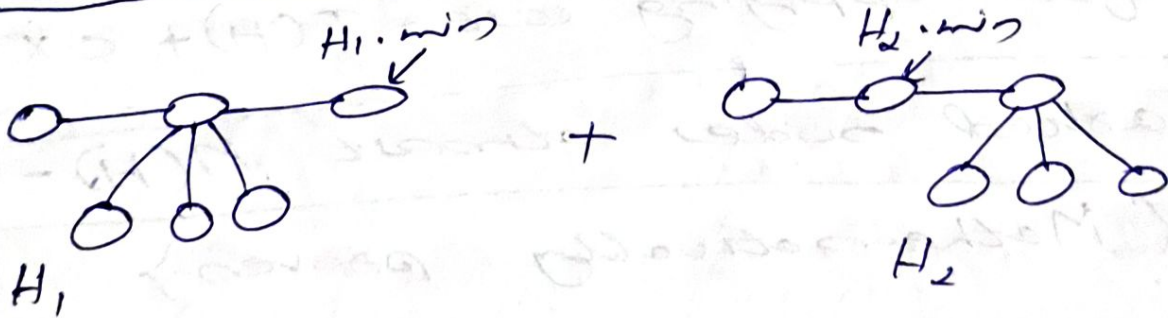
$$\therefore \text{Amortized cost} = \text{Actual cost} + \text{change in potential} \\ = O(1) + 4 \cdot C = \underline{\underline{O(1)}}$$

6/11/2025
Thursday

> Union of two Fibonacci Heaps

> Steps

- ① Concatenate rootlist of H_1 and H_2 and create a new heap, H .
- ② Determine new 'min' key node from the merged fibonacci heap.



If $H_1.min < H_2.min$,
 $H.min = H_1.min$

If $H_2.min < H_1.min$,
 $H.min = H_2.min$

ALGORITHM

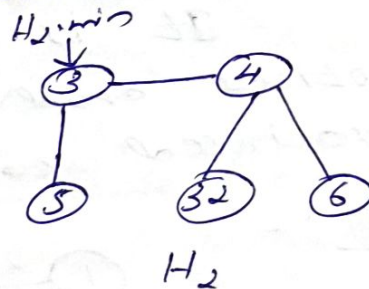
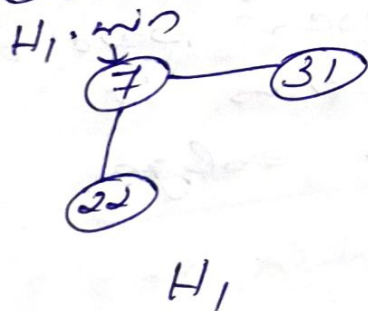
7

FIB-HEAP-UNION(H_1, H_2)

- 1) $H = \text{MAKE-FIB-HEAP}()$
- 2) $H.\text{MIN} = H_1.\text{MIN}$
- 3) Concatenate rootlist of H_2 with the rootlist of H_1 .
- 4) If $(H_1.\text{MIN} = \text{NIL})$ or $(H_2.\text{MIN} \neq \text{NIL}$ and $H_2.\text{MIN}.\text{key} < H_1.\text{MIN}.\text{key})$
- 5) $H.\text{MIN} = H_2.\text{MIN}$
- 6) $H.N = H_1.N + H_2.N$
- 7) return H

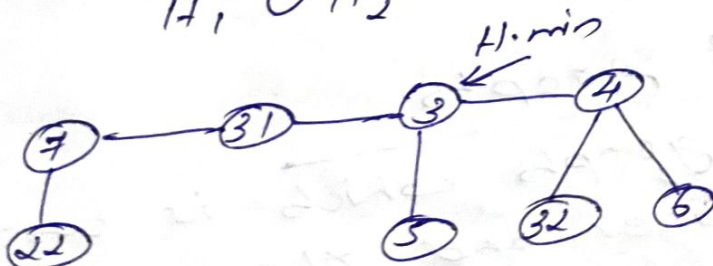
Example

Consider two fibonacci heaps:-



Union of H_1 and H_2

$H_1 \cup H_2$



Here,
 $H_2.\text{min} < H_1.\text{min}$
 $3 < 7$

$\therefore H.\text{min} = 3$